Chapter 15

High assurance software architecture and design

Muhammad Ehsan Rana[a] and Omar S. Saleh[b]
[a]*Asia Pacific University of Technology & Innovation (APU), Technology Park Malaysia, Kuala Lumpur, Malaysia,* [b]*Studies, Planning and Follow-Up Directorate, Ministry of Higher Education and Scientific Research, Baghdad, Iraq*

15.1 Introduction

When a software system is constructed, it needs to be carefully architected and designed to get benefited from the inherent features of software quality. In addition to hardware, which is usually taken as the primary source for improving performance, design and architecture have a high impact on the performance of a software application. Software design impacts various aspects of software quality including reusability, maintainability, flexibility, reliability, and efficiency. Software architecture ensures the presence of quality in a software application by implementing structures such as architectural and design principles and patterns and by avoiding antipatterns while applying these constructs. Studies indicate improvements in quality factors using design and architectural refinements. Accordingly, for the application that lacks these quality attributes, it can be redesigned or refactored to incorporate these features of software quality.

15.2 Software architecture patterns

Well-designed architecture helps software applications to scale, better maintain, and respond to increased changing requirements. Software architecture patterns define an application's fundamental characteristics and behavior that make it scalable, modular, and maintainable. Without a standard architecture in mind, developers usually are lost in the code's complexity, especially when the application size gets large. It eventually creates unorganized codes and a lack of coherence among various components of an application [1]. This section discusses the three most used software development architectures: client-server pattern, layered pattern, and model-view-controller pattern. Each of these architectural patterns follows the principle of divide and conquer and separation of concerns.

System Assurances. https://doi.org/10.1016/B978-0-323-90240-3.00015-1

15.2.1 Client-server pattern

Most of the implemented web applications today are based on client-server architecture. This pattern divides a software application into two main areas, where one acts as the client and the other as the server. In the client-server architecture pattern, a client sends the request through a web browser, and the server responds based on the request received from the client, as illustrated in Fig. 15.1. In this pattern, the client needs to know the server's address, whereas the server does not need to know the client's location. In some cases, the client can be used as a server and the server as a client. Some of the standard protocols that the client and server are associated with include the File Transfer Protocol (FTP), Simple Mail Transfer Protocol (SMTP), and Hypertext Transfer Protocol (HTTP) [2,3].

15.2.1.1 Components of client-server architecture

The components of client-server architecture include:

a. Client: the requester of the processes either through a web browser interface or chat client, email client, etc.

b. Server: the receiver of the requests from the clients. It processes the requests, gathers the required information, generates the results, and sends them back to the client. Once the server completes processing the client request, it tackles other requests such as the FTP server, chat server, and database server.

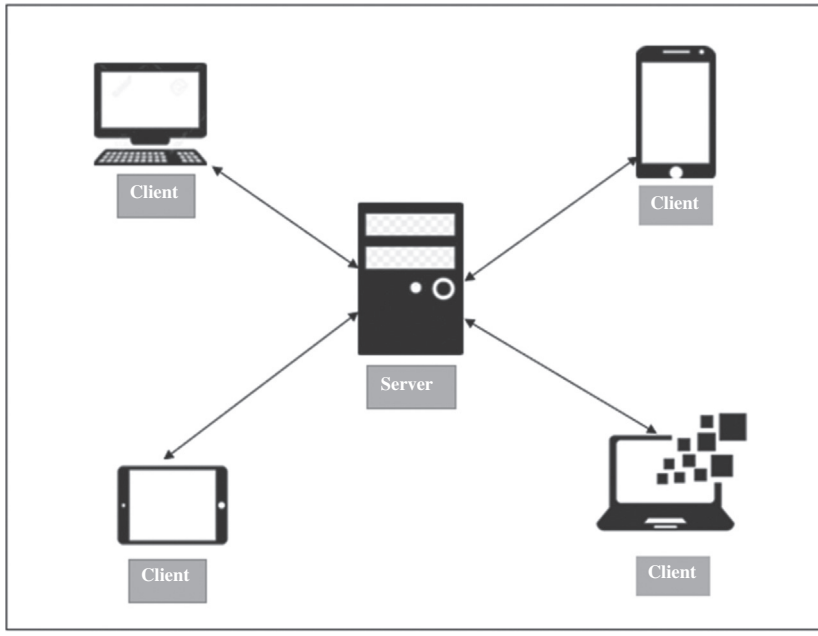

FIG. 15.1 Client-server model.

15.2.1.2 Advantages of client-server architecture

The client-server architecture-based design has several benefits that include:

a. Easy maintenance is the primary advantage of the client-server pattern. The roles are distributed among several standalone systems; hence, maintenance of the clients is independent.
b. The client-server architecture promotes increased scalability. The application scalability is one of the significant advantages considering the latest application development trends as the demand for technological improvement is rapidly increasing.
c. Data is centralized in the client-server architecture, where multiple clients can access it from various locations. The possibility of shared resources and services makes it easier to manage, modify, and reuse software modules [4].

15.2.2 Layered architecture pattern

The layered architecture pattern is among the most common architectural patterns used at the enterprise level, consisting of a hierarchy of layers, as shown in Fig. 15.2. The services communicate between these layers via ports. A layer is modeled as a relation between used and provided services, whereas a layer composition is defined by means of relational composition [5].

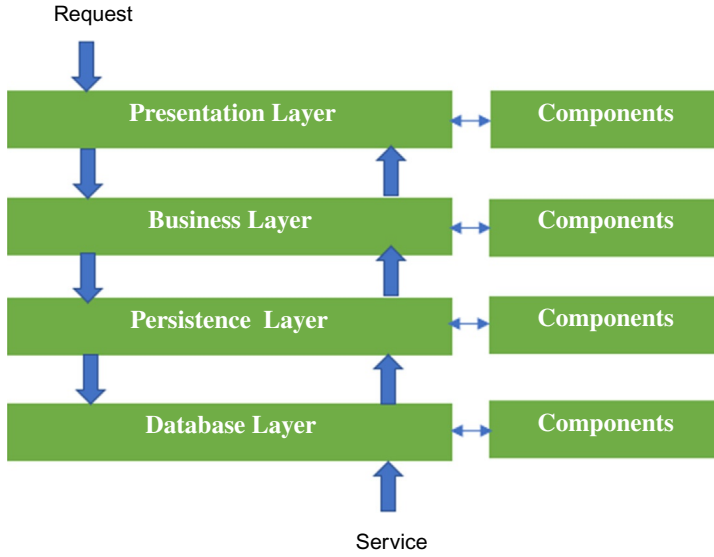

FIG. 15.2 Layered architecture pattern.

15.2.2.1 Components of layered architecture

The layered architecture pattern helps encapsulate and decouple the application by grouping the distinct segments using their functional role. This pattern follows the separation of concerns principle, resulting in improved testability and maintainability of the application. However, involving too many layers may deteriorate performance. Hence, it is advisable to limit the number of layers to approximately four. The four main layers utilized in the layered architecture pattern consist of the presentation layer, the business layer, the persistence layer, and the database layer.

a. **Presentation layer:** The presentation layer is the user interface (UI) layer representing the website or application layout, including web pages, forms, and interactions of APIs.
b. **Business layer:** The business layer contains the application's business logic, including the security, access, and authentication tasks. This layer is responsible for performing specific business rules associated with the user requests coming from the presentation layer.
c. **Persistence layer:** The persistence layer is liable for data presentation. It includes the data access object, object relational mappings, and other modes of presenting persistent data at the application level [6].
d. **Database layer:** The database layer is responsible for storing application data and retrieving it.

15.2.2.2 Advantages of layered architecture

The layered architecture-based design has numerous benefits that include [7,8]:

a. Layers are separated from one another, allowing developers to manage and maintain them independently. It helps in simplifying the application infrastructure as each layer is isolated.
b. Each layer manages a single aspect of the application, making it easier for managing the code. Layers can also be independently tested.
c. The layered architecture approach is particularly good for cloud-based applications that require a flexible and portable architecture.
d. This approach can also be applied to manage legacy systems as the application's architecture can be broken into separate independent modules.
e. The layered approach also provides a swift web solution without much complexity.

15.2.3 Model-view-controller (MVC) pattern

The model-view-controller (MVC) pattern is an excellent example of separation of concerns in an object-oriented paradigm. It divides the application into three logical components, the model, the view, and the controller, as shown in Fig. 15.3. The approach is now widely used in web applications. The MVC pattern ensures the rule of low coupling and high cohesion. The MVC architecture

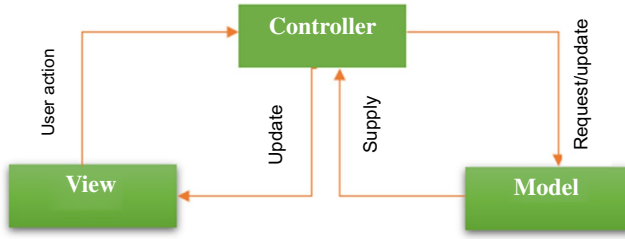

FIG. 15.3 Model view controller pattern.

may be difficult to understand at first but offers several benefits to the architects and designers. The basic idea of MVC is to divide the application into three interconnected components to separate the logic using the ways information is presented to and accepted by the client [9].

15.2.3.1 Components of model-view-controller (MVC) architecture

The model-view-controller (MVC) pattern is a three-layered pattern. The first layer is for user input logic, the second layer is for business logic, and the third is for user interface logic [10,11].

a. **Model:** The model represents the data and the core functionality of an application. It is commonly used to insert, retrieve, or update data into the database associated with the application.
b. **View:** As the name represents, the view deals with how the data is presented on the user interface. Users can interact with the application components through this interface.
c. **Controller:** The controller is responsible for handling the user requests and acts as an intermediator between the model and view.

15.2.3.2 Advantages of model-view-controller (MVC) architecture

The MVC architecture-based design offers several benefits that include [9,12]:

a. The MVC architecture helps to manage the software complexity issues by dividing the processes into three separate components.
b. As it does not use server-based forms, developers will have complete control over the application behavior.
c. The MVC architecture provides the ability to present multiple views for the model. It is an essential feature of contemporary applications as the demand for new ways of accessing the application increases day by day.
d. MVC-based applications are highly maintainable as each component can be managed independently.
e. MVC is instrumental in creating extensible and scalable applications.
f. The reusability and testability of the components can also be increased by applying the MVC pattern.

15.3 Software design principles

Object-oriented design principles provide established guidelines to construct quality solutions. Object-oriented experts and evangelists have identified numerous principles and design heuristics. However, five of these are considered significantly crucial for an object-oriented design [13]. These five design principles include Single Repository Principles, Open-Closed Principle, Liskov Substitution Principle, Interface Segregation Principle, and Dependency Inversion Principle. These principles are collectively termed as SOLID considering the first letter of their names. Following is a brief description of these principles.

15.3.1 Single responsibility principle

The single responsibility principle (SRP) states that each system module should only serve one and only one actor. The most important thing in this principle is the separation of concerns. Each actor should have his/her own implementation stored in different classes. This principle is useful to separate the concerns and perform any changes if necessary.

15.3.2 Open-closed principle

The open-closed principle (OCP) refers to the software entities and artifacts that are open for extension and closed for modification. This principle can be applied using different levels of abstraction (using interfaces) in order to protect different modules from the changes.

15.3.3 Liskov substitution principle

The Liskov substitution principle (LSP) refers to the ability of a subtype to replace the parent type without affecting the intended operation. For example, if a billing application contains a license type, this type can be substituted by a personal license type and a business license type. In other words, the billing application respects the Liskov substitution principle as it does not depend on the subtypes, but only uses the parent type which is license [13].

15.3.4 Interface segregation principle

The interface segregation principle (ISP) refers to removing unnecessary dependencies by different classes. This principle is followed when a class containing different implementations for different components is split into specific interfaces for each component. It is useful as it helps to prevent recompilation and redeployment of all components if a change occurs.

15.3.5 Dependency inversion principle

The dependency inversion principle (DIP) refers to the separation of the high-level modules from low-level ones. This principle can be applied if the components depend only on abstractions (interfaces) and not on the concrete implementations.

15.4 Software design patterns

Design patterns are object-oriented software design practices for solving common design problems [14]. Software architecture often appeared as a set of classes that can become too complicated to be easily understood. These complexities include complicated interactions and coupling between objects within the software system. Design patterns may reduce these complexity problems by offering proven solutions. As design patterns are tested and reusable solutions for reoccurring problems, they act as pre-made blueprints customized to solve these design issues [15].

15.4.1 Essential elements of design patterns

A design pattern consists of four essential elements that include the pattern's name, problem, solution, and consequences of using that pattern, as shown in Fig. 15.4 [16].

A pattern name should describe the pattern's problem, solution, and consequences in a word or two. It is essential as it provides a high level of abstraction and makes it easier to think and communicate design with others. Taking the Mediator design pattern, for instance, it is visible from its name that it creates an object to act as a mediator to manage complex communication between objects; thus, the term Mediator is devised.

A problem decides the usability of a pattern as it provides context for the pattern, similar to a set of conditions for the pattern to be applied. This set of conditions determines the applicability of a pattern to an existing problem [17]. In the Mediator design pattern, one of the conditions to apply the said

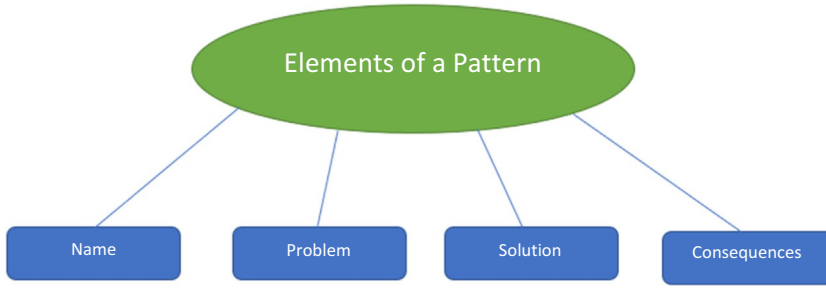

FIG. 15.4 Essential elements of a design pattern [16].

pattern is when communications between objects are defined but in complex ways, resulting in unstructured interdependencies between objects that are hard to comprehend. One of the primary examples of such a situation would be the presence of high coupling in a given system. If a given system is highly coupled, implementing the Mediator design pattern is recommended as it reduces the unstructured interdependencies and complex communications between objects.

A solution is not a concrete explanation of how the pattern should be applied. It explains the elements that make up the design, such as their relationships, responsibilities, and collaborations [16]. Thus, instead of explaining the pattern in a concrete manner, the pattern itself acts as a template in which it can be applied in various situations by providing a description of how the elements should be arranged to solve a given problem.

The consequences of a pattern include the description of the pros and cons when applying this pattern [16]. Studies suggest that not all designs provide the best solution [18]. Thus, one must consider the consequences of applying the pattern as they are critical in evaluating other alternatives present in the body of knowledge. The consequences of a given pattern can impact the system's quality attributes (extensibility, reusability, portability, etc.). Listing the consequences of each pattern and comparing them may assist in deciding the best pattern for a specific problem.

15.4.2 Classification of design patterns

Since software design is a challenging phase in the software development life cycle, many design problems can frequently occur in every project. Therefore, design patterns come in place to overcome these reoccurring design problems. Design patterns are well-tested solutions to many well-known design problems. The design patterns are also categorized into three main categories, which are creational design patterns, structural design patterns, behavioral design patterns. Each design pattern category contains many design patterns that can be used depending on the problem and situation [19].

15.4.2.1 Creational design patterns

The creational design patterns are responsible only for the creation of the objects depending on the situation. These patterns give different ways of creating objects to reduce the complexity of the design and creation mechanisms [20].

15.4.2.2 Structural design patterns

The structural design patterns simplify the design and make relationships between different system's components to have different functionalities [20].

15.4.2.3 Behavioral design patterns

Since communication is essential in software systems, the behavioral design patterns serve to simplify communication and produce more flexibility while communicating [20].

15.4.3 Gang of four design patterns

Gamma et al. [16] provide a catalog of 23 design patterns with their detailed descriptions including classification, intent, motivation, applicability, structure, implementation, and consequences. These design patterns are commonly referred to as gang of four (GoF) design patterns. A concise introduction of these GoF design patterns is presented in Table 15.1, which exhibits the verity and versatility of these patterns for various design problems.

15.4.4 Benefits of using design patterns

In a typical software development project, many changes occur throughout the development process. It is arduous to ensure that the changes made would not introduce new defects into the system as there are way too many uncertainties. However, design patterns are remarkably flexible to the changes as they are tested solutions to the problems that highly mitigate the risk of emerging new defects. Some common benefits of using design patterns are listed below:

a. Design patterns are able to reduce the development time for a software project [21].
b. Design pattern can be used to develop more secure and more reliable software systems.
c. With the tested, proven development paradigm and reusable solution for recurring problems, the system development process is accelerated and the software quality is improved.
d. Design patterns offer a highly stable solution for design problems. As design patterns provide tested and proven solutions for specific problems, the solutions they offer are comparatively very stable and free from bugs [22].
e. Since the design has been proven to work, it saves time during the design phase and reassures the quality of the system design. The probability of defects being met along the development life cycle is minimized as the project has been designed and tested for the long run [16].
f. Through the pattern's high-level abstraction, designers can communicate effectively with other designers to discuss the pattern that is best suited for the project given. This saves more time and costs to be allocated for the design phase.

TABLE 15.1 Gang of four (GoF) design patterns.

Category	Design pattern	Description
Creational	Abstract factory	Create instances for families of classes
	Builder	Separates construction process from representation
	Factory method	Defer instantiation to sub-classes
	Prototype	Creates clone of a class to produce a new object
	Singleton	Restricts object creation to a single instance only
Structural	Adaptor	Convert the interface of a class into another interface client expects
	Bridge	Decouple an abstraction from its implementation
	Composite	Compose objects into tree structures to represent part-whole hierarchies
	Decorator	Provides a way to dynamically modify the behavior of an object without sub-classing
	Façade	Provide a unified interface to a set of interfaces in a subsystem
	Flyweight	Addresses sharing to support large numbers of fine-grained objects efficiently
	Proxy	Provides placeholder for another object to control access to it
Behavioral	Chain of responsibility	An object takes on the job of finding which object can satisfy a client's request
	Command	Encapsulates request as an object
	Interpreter	Defines representation for language elements and interpret it with an interpreter object
	Iterator	Allows traversing an aggregate object sequentially
	Mediator	Define an object that encapsulates how a set of objects interact to promote loose coupling
	Memento	Saves internal state of an object without violating encapsulation
	Observer	Notifies the dependent objects automatically when the object changes state
	State	Distributes state specific logic across classes that represent an object's state
	Strategy	Allows an algorithm to vary independently from the clients that use it
	Template method	Defines skeleton of an algorithm in a method
	Visitor	Defines new operations without changing the class

15.5 Software design antipatterns

Design antipatterns are the bad design practices and the weak solutions used when building a software system [23]. Software quality can be directly affected by design, either positively or negatively. The design patterns positively impact the software quality, while the design antipatterns negatively affect the software quality. Antipatterns are generally caused either by lack of experience or by using the design patterns in the incorrect place [24].

Software quality is an imperative concern for many software development companies as clients want the acquired software system to be agile and perfect. Different quality factors or attributes must be taken into consideration in order to attain high-quality software. Some of the crucial quality attributes include performance, reusability, testability, and reliability. Many of these attributes are negatively impacted by the high complexity of the software. Software performance is described by the ability of the software system to be stable and responsive in a particular environment and workload [25]. Reusability is the ability to efficiently reuse previous software components, design, and implementation in the new development projects [25]. Reliability refers to the ability of a software system to work without any failure in a specified environment within a specific period [25]. Software complexity refers to the high level of interactions among the internal structures of the software system. The complexity is generally related to the software design as the coupling between the system's modules gradually increases due to the interconnected links between the different classes, which negatively affects the software system's quality [26].

15.5.1 Some common antipatterns

The following is a brief description of some of the common design antipatterns.

15.5.1.1 Blob antipattern

The Blob antipattern is a bad practice in the design that refers to the use of one big class to manage and perform many operations. The Blob antipattern reduces the cohesion of the software system and dominates all the processing and decision-making in the software [27]. This antipattern is considered bad practice as it violates the object-oriented design principles, eliminates the reusability, and complicates the testability of the class. Moreover, it consumes many resources for simple operations as the class is entirely loaded into the system's memory [24].

15.5.1.2 Spaghetti Code antipattern

The Spaghetti Code is a design antipattern that refers to the misuse of object-oriented principles. This antipattern consists of using methods with lengthy implementations and no parameters but is only limited to the usage of global

variables [28]. In general, the Spaghetti Code eliminates important object-oriented concepts like inheritance and polymorphism [29].

15.5.1.3 Poltergeist antipattern

The Poltergeist, also known as Gypsy Wagons, is an antipattern that can be found on systems containing classes with a short life cycle and minimal operations and responsibilities [30]. These Poltergeist antipatterns are usually used to invoke some other system's classes or methods and waste the system's resources as long as they are used [24].

15.5.1.4 Functional decomposition antipattern

Functional decomposition is an antipattern that consists of writing code without thinking in an object-oriented manner. This antipattern lets classes look like a function with a single purpose. The classes perform one single operation with all the data attributes that are private and used only inside the class. This antipattern also eliminates the important object-oriented concepts like inheritance and polymorphism, which results in reduced reusability [28].

15.5.1.5 Swiss Army Knife antipattern

The Swiss Army Knife antipattern refers to the class that contains many responsibilities. This antipattern is characterized by a large number of methods and interface implementations [28].

15.5.2 When design patterns turn into antipatterns

Developers need to recognize the trade-offs between design factors when applying the patterns to the design. Generally speaking, a class having more than one role or responsibility violates the single responsibility principle as it potentially causes low cohesion and complicates the process of fixing vulnerabilities. However, some of the design classes with one role still negatively impact the system's integrity. For instance, when the composite pattern is applied, it affects the integrity of the software system as it can potentially make the design overly general, which increases the security problems. Therefore, using the composite pattern indicates that the user values transparency more than security [16].

Similarly, the Singleton design pattern also impacts applications' overall integrity as it violates the open-closed principle. Singleton ensures one class has only one object instantiated, indicating that it does not allow inheritance as a private constructor is used. When inheritance is avoided, the open for extension principle will be violated. Changing the existing code that functions perfectly sounds risky, and it might cause the emergence of new threats and bugs which the attacker can exploit to perform any unauthorized access. Besides, allowing inheritance in the Singleton class does not make sense as when inheritance is permitted, there will be more than one instance for the class, making

the class not a Singleton class anymore. Moreover, by using a reflection attack, the constructor of the Singleton class can be altered from private to public, indicating that the constructor can be called from outside, which then increases the risk of emerging new threats.

15.6 Conclusion

This chapter discusses the fundamental building blocks of software architecture that include software architecture patterns, software design principles, software design patterns, and antipatterns. Among the primary steps for refinement of software design is its conversion in a more dynamic, adaptable, and flexible format. These building blocks play a pivotal role in improving software quality and avoiding practices that generate a faulty design. Design principles outline the most common scientifically derived approach for constructing a flexible and robust design. The use of design principles provides established guidelines to produce quality solutions. Design patterns rely on proven object-oriented design approaches to construct reliable and reusable solutions for the associated design problems in a system. One of the key purposes of most design patterns is to create a simple solution that is extendible and loosely coupled. Hence, the design produced as a result of applying design patterns eventually reduces the interdependencies between program components and makes them easier to extend and maintain. Software architecture patterns represent a more abstract layer of software design than design patterns and define an application's fundamental characteristics and behavior that makes it scalable, modular, and maintainable. Antipatterns identify the improper application design, which leads to increased development time and can become a bottleneck for the application's performance. They provide a legitimate way to document the problem scenarios and their solutions which assists the designers in watching out for similar situations to avoid bugs and other design constraints that hinder performance. In typical software development, these architectural and design constructs need to be applied in sequence to ensure quality software generation. Therefore, if these three elements of object orientation are not followed sequentially, it increases the chance of inducing flaws and defects in the design.

References

[1] M. Richards, Software Architecture Patterns, first ed., O'Reilly Media, Inc., 2015.
[2] H.S. Oluwatosin, Client-server model, J. Comput. Eng. 16 (1) (2014) 67–71.
[3] A. Sharma, M. Kumar, S. Agarwal, A complete survey on software architectural styles and patterns, Procedia Comput. Sci. 70 (2015) 16–28, https://doi.org/10.1016/j.procs.2015.10.019.
[4] A. Senson, A. Burton, T. Boulanger, Software Architecture & Software Design Patterns for Startups, RIC Centre, 2021. https://riccentre.ca/software-architecture-design-patterns/.
[5] D. Marmsoler, A. Malkis, J. Eckhardt, A model of layered architectures, in: Formal Engineering Approaches to Software Components and Architectures (FESCA'15), 2015, pp. 47–61, https://doi.org/10.4204/EPTCS.178.5.

[6] A. Wickramarachchi, Software Architecture Patterns. Layered Architecture | by Anuradha Wickramarachchi | Towards Data Science, 2017. https://towardsdatascience.com/software-architecture-patterns-98043af8028 (Accessed 22 November 2021).
[7] V.S. Sharma, P. Jalote, K.S. Trivedi, Evaluating performance attributes of layered software architecture, in: G.T. Heineman, I. Crnkovic, H.W. Schmidt, J.A. Stafford, C. Szyperski, K. Wallnau (Eds.), Component-Based Software Engineering. CBSE 2005, Lecture Notes in Computer Science, vol. 3489, Springer, Berlin, Heidelberg, 2005, https://doi.org/10.1007/11424529_5.
[8] C. Liyan, Application research of using design pattern to improve layered architecture, in: 2009 IITA International Conference on Control, Automation and Systems Engineering (case 2009), IEEE, 2009, July, pp. 303–306.
[9] M.U. Khan, T.V. Rao, XWADF: architectural pattern for improving performance of web applications, Int. J. Comput. Sci. 11 (2) (2014) 105–113.
[10] T. Dey, A comparative analysis on modeling and implementing with MVC architecture, in: IJCA Proceedings on International Conference on Web Services Computing (ICWSC), vol. 1, 2011, November, pp. 44–49.
[11] A. Leff, J.T. Rayfield, Web-application development using the model/view/controller design pattern, in: Proceedings Fifth IEEE International Enterprise Distributed Object Computing Conference, IEEE, 2001, September, pp. 118–127.
[12] A. Majeed, I. Rauf, MVC architecture: a detailed insight to the modern web applications development, J. Sol. Photoenergy Syst. 1 (1) (2018) 1–7.
[13] R.C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design, Prentice Hall, 2018.
[14] M.O. Onarcan, Y. Fu, A case study on design patterns and software defects in open source software, J. Softw. Eng. Appl. (2018) 249–273, https://doi.org/10.4236/jsea.2018.115016.
[15] A. Shvets, Dive into Design Patterns, 2019. Available at: https://www.goodreads.com/book/show/43125355-dive-into-design-patterns (Accessed 22 November 2021).
[16] E. Gamma, R. Helm, J. Ralph, J. Vlissides, Design Patterns—Elements of Reusable Object-Oriented Software, Addison-Wesley Professional, 1995.
[17] X. Ferre, N. Juristo, A. Moreno, I. Sánchez, A software architectural view of usability patterns, in: 2nd Workshop on Software and Usability Cross-Pollination (at INTERACT'03) Zurich (Switzerland), 2003, September.
[18] P. Wendorff, ASSET GmbH, Assessment of design patterns during software reengineering : lessons learned from a large commercial project, IEEE Xplore (2001) 77–84.
[19] S. Hussain, J. Keung, A.A. Khan, Software design patterns classification and selection using text categorization approach, Appl. Soft Comput. 58 (2017, September) 225–244, https://doi.org/10.1016/j.asoc.2017.04.043.
[20] A. Anand, G. Bansal, Interpretive structural modelling for attributes of software quality, J. Adv. Manag. Res.14 (3) (2017) 256–269, https://doi.org/10.1108/JAMR-11-2016-0097.
[21] R. Subburaj, G. Jekese, C. Hwata, Impact of object oriented design patterns on software development, Int. J. Sci. Eng. Res. 6 (2) (2015) 961–967.
[22] A. Ampatzoglou, G. Frantzeskou, I. Stamelos, A methodology to assess the impact of design patterns on software quality, Inf. Softw. Technol. 54 (4) (2012) 331–346, https://doi.org/10.1016/j.infsof.2011.10.006.
[23] M. Abidi, F. Khomh, Y. Guéhéneuc, Anti-patterns for multi-language systems, in: Proceedings of the 24th European Conference on Pattern Languages of Programs—EuroPLop '19, 2019, pp. 1–14, https://doi.org/10.1145/3361149.3364227.
[24] S. Lujan, F. Pecorelli, F. Palomba, A.D. Lucia, V. Lenarduzzi, A Preliminary Study on the Adequacy of Static Analysis Warnings with Respect to Code Smell Prediction, MaLTeSQuE, September 2020.

[25] A. Anand, G. Bansal, Interpretive structural modelling for attributes of software quality, J. Adv. Manag. Res. 14 (3) (2017, August) 256–269, https://doi.org/10.1108/JAMR-11-2016-0097.
[26] N. Vanitha, R. ThirumalaiSelvi, SICCAT: software inheritance coupling complexity analysis tool, Int. J. Eng. Tech. 4 (2) (2018) 62–67. Available at: http://www.ijetjournal.org.
[27] S. Hussain, et al., Methodology for the quantification of the effect of patterns and anti-patterns association on the software quality, IET Softw. 13 (5) (2019) 414–422, https://doi.org/10.1049/iet-sen.2018.5087.
[28] N. Vavrová, V. Zaytsev, Does Python smell like Java? Tool support for design defect discovery in Python, Art Sci. Eng. Program. 1 (2) (2017, April), https://doi.org/10.22152/programming-journal.org/2017/1/11.
[29] S. Lujan, F. Pecorelli, F. Palomba, A.D. Lucia, V. Lenarduzzi, A Preliminary Study on the Adequacy of Static Analysis Warnings with Respect to Code Smell Prediction A Preliminary Study on the Adequacy of Static Analysis Warnings with Respect to Code Smell Prediction, MaLTeSQuE, 2020, September.
[30] S.R.A. Al-Rubaye, Y.E. Selcuk, An investigation of code cycles and Poltergeist anti-pattern, in: 2017 8th IEEE International Conference on Software Engineering and Service Science (ICSESS), November, vol. 2017, 2017, pp. 139–140, https://doi.org/10.1109/ICSESS.2017.8342882.